

Design and Implementation of a Pipelined Wallace Tree Multiplier using Verilog HDL

1. Cover Page

Project Title:

Design and Implementation of a Pipelined Wallace Tree Multiplier

Submitted by: Arun Gopalakrishna Iyer

Name: Register No: 23150182 (Enrollment_No)

Course: Certificate Course in VLSI Design

Date: 28-Jun-2026

2. Abstract

A multiplier is one of the most critical arithmetic units in digital systems such as processors, DSP processors, AI accelerators, and communication systems. The performance of a multiplier directly affects the overall system speed.

This project focuses on the design and implementation of a **pipelined Wallace Tree multiplier** using Verilog HDL. The Wallace Tree architecture reduces multiplication delay by using parallel partial product reduction through Full Adders and Half Adders.

Pipeline registers are introduced between different stages to improve the operating frequency and throughput of the multiplier.

The design is verified using a self-checking testbench with multiple random test cases.

3. Objectives

The objectives of this project are:

- Understand the Wallace Tree multiplier architecture.
- Generate partial products using AND gates.
- Implement partial product reduction using compressors.
- Design pipeline stages for improved performance.
- Develop Verilog RTL implementation.
- Verify functionality using simulation.
- Analyze latency and throughput.

4. Design Specification

Parameter Specification

Input Width 8-bit Inputs A[7:0], B[7:0] Output 16-bit Product Architecture Wallace Tree Pipeline 3/4 Stage Pipeline Language Verilog HDL Reset Active Low Simulation Tool VCS / ModelSim Target 50 MHz Frequency

5. Wallace Tree Multiplier Theory

A Wallace multiplier is a hardware implementation of a binary multiplier, a digital circuit that multiplies two integers. . It condenses multiple rows of partial products into just two rows (Sum and Carry) by passing sets of 3 bits of equal weight through 1-bit full adders, outputting 2 bits of higher weight. A 3:2 compressor is essentially a standard Full Adder. It takes three inputs of the same positional weight and compresses them into two outputs

5.1 Basic Multiplication

The basic multiplication generates all 8 partial products.

	a7 a6 a5 a4 a3 a2 a1 a0	
	b7 b6 b5 b4 b3 b2 b1 b0	

	a7b0.....a0b0.	=> pp0
	a7b1.....a0b1	=> pp1
	a7b2.....a0b2	=> pp2
	a7b3.....a0b3	=> pp3
	a7b4.....a0b4	=> pp4
	a7b5.....a0b5	=> pp5
	a7b6.....a0b6	=> pp6
	a7b7.....a0b7	=> pp7

5.2 Wallace Tree Reduction

In reduction step each 3 rows (pp0..pp2 & pp3...pp5) is reduced to create 2 rows of sum and carry

	a7b0.....a0b0.	=> pp0
	a7b1.....a0b1	=> pp1
	a7b2.....a0b2	=> pp2

	S7.....S0.	=> row1_0
	c7.....c0.	=> row1_1
	a7b3.....a0b3	=> pp3
	a7b4.....a0b4	=> pp4
	a7b5.....a0b5	=> pp5

	S7.....S0.	=> row1_2
	c7.....c0.	=> row1_3
	a7b6.....a0b6	=> pp6 =>
row1_4		

```
a7b7.....a0b7
row1_5
```

=> pp7 =>

Now keep adding the 6 rows (row1_0 ...row1_6) from stage 1 down to just 2 rows

```
row_1_0
row_1_1
row_1_2
-----
row_2_0 (Sum)
row_2_1 (Carry)
```

```
row_1_3
row_1_4
row_1_5
-----
row_2_2 (Sum)
row_2_3 (Carry)
```

Now reduce 4 rows to 2 rows and get final product row_2_0 (Sum) row_2_1 (Carry) row_2_2 (Sum) -----
----- row_3_0

```
row_2_3 (Carry)
-----
final product
```

6. Proposed Architecture

Block Diagram

A	B
Partial product generation	
Addition of partial products	
Addition of final row partial products	
Final Product	

7. Pipeline Architecture

The multiplier is pipelined into 3 stages using registers

Stage 1

Make all 8 partial products and add some of them together. After this stage there will be 6 rows, first two set of 3 rows added and last two partial product rows

Stage 2

In this stage we keep adding the 6 rows from stage 1 down to just 2 rows

Stage 3

Final addition of the last 2 rows to get the answer

8. RTL Design

Project directory:

Wallace_Multiplier/

|

|-- src/rtl/

- ||| |-- wallace_mult_8bit.v | |-- tb_wallace_mult_8bit.v | |-- tb_rand_wallace_mult_8bit.v |

- | -- out/ => Simulation outputs

| -- doc/ => documentation & reports

9. RTL Module Description

NA . No Sub modules used.

10. Testbench Description

The testbench performs:

- Clock generation
- Reset sequence
- Random input generation
- Output checking

Test Cases

Simple Testing: applies some test values and prints the result.

testbench file : tb_wallace_mult_8bit.v

Random testing: repeat(1000) Generate random A,B Compare: RTL output vs A*B reference.

testbench file : tb_rand_wallace_mult_8bit.v

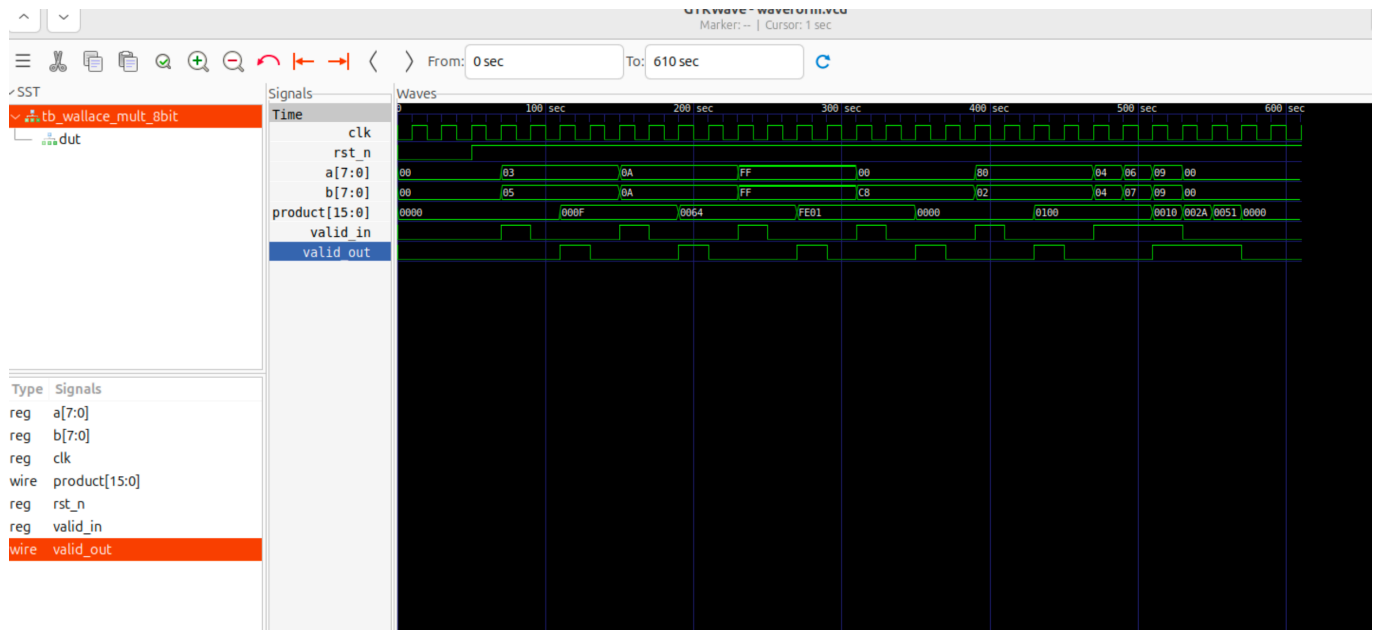
11. Simulation Results

Included:

out/tb_wallace_mult_8bit.out & out/tb_rand_wallace_mult_8bit.out

Waveform Screenshot

Show:



12. Verification Result

Run & Output: iverilog -o out/tb_wallace_mult_8bit.out tb_wallace_mult_8bit.v wallace_mult_8bit.v && vvp out/tb_wallace_mult_8bit.out

VCD info: dumpfile waveform.vcd opened for output. Starting tests... Time=130 -> product = 15 Time=210 -> product = 100 Time=290 -> product = 65025 Time=370 -> product = 0 Time=450 -> product = 256 Time=530 -> product = 16 Time=550 -> product = 42 Time=570 -> product = 81 Tests done. Check the printed products above against: Expected: 15, 100, 65025, 0, 256, then 16, 42, 81
tb_wallace_mult_8bit.v:120: \$finish called at 610 (1s)

Run & Output: iverilog -o out/tb_rand_wallace_mult_8bit.out tb_rand_wallace_mult_8bit.v wallace_mult_8bit.v && vvp out/tb_rand_wallace_mult_8bit.out

Starting 1000 random tests... All 1000 random cases passed. Execution finished at time 60070 ns Completed 1000 cases in 60070.000 ns Throughput = 16.647 MOPS/s Throughput = 33.294 MB/s
tb_rand_wallace_mult_8bit.v:119: \$finish called at 60070 (1s)

Test Result Basic multiplication PASS Random test PASS Maximum input PASS Zero input PASS

13. Performance Analysis

Throughput

Throughpt calculated on basis of random tests of 1000 operations Completed 1000 cases in 60070.000 ns Throughput = 16.647 MOPS/s Throughput = 33.294 MB/s

14. Comparison

TBD. Based on any benchmarks available.

15. Conclusion

The current design in `wallace_mult_8bit.v` is a good educational implementation, and several improvements would make it more efficient and synthesis-friendly.

16. Future Enhancements

1. Convert the module to support any bit-width, such as 8, 16, 32, etc. This makes the same architecture reusable instead of being hard-coded for 8 bits.
2. Upgrading to higher-order compressors (e.g., 4:2, 5:2 compressors) dramatically decreases the number of reduction stages and interconnect routing.
3. The last stage currently uses simple addition. A carry-lookahead adder or a faster final adder can reduce critical path delay.
4. Support signed multiplication. Add support for signed operands with sign extension logic.